

CSEN1131 - SOFTWARE ENGINEERING

UNIT-I

Different Definitions of Software Engineering:

- ✓ Software Engineering is the process of solving customers' problems through the systematic development and evolution of large, high-quality software systems within cost, time, and other constraints.
- ✓ Software Engineering is a systematic approach to the development, operation, maintenance, and retirement of software.
- ✓ Software Engineering is the application of science and mathematics by which the capabilities of computer equipment are made useful to man via computer programs, procedures, and associated documentation.

The story of Software development and issues faced

The story of software development is a fascinating journey that has evolved over several decades. It has transformed from simple, isolated programs to complex, interconnected systems that power almost every aspect of our daily lives. Along this journey, developers have faced numerous challenges and issues, shaping the field into what it is today. Here's a brief overview:

1. Early Days (1940s-1960s):

- **Hardware Limitations:** Early computers had limited processing power and memory. Developers had to write efficient code to make the most of these resources.
- **Lack of Standards:** There were no standardized programming languages, making it challenging to write portable code that could run on different machines.

2. Birth of High-Level Languages (1950s-1970s):

- **Assembly Language:** Initially, programming was done in assembly language, which was machine-specific and tedious.
- **High-Level Languages:** Fortran, Lisp, and COBOL were among the first high-level languages, making programming more accessible.

3. Software Crisis (1960s-1970s):

- **Increasing Complexity:** As software systems became larger and more complex, developers faced challenges in managing and maintaining them.
- **Poor Quality:** The rapid growth of software led to quality issues, with bugs and errors becoming more prevalent.

4. Rise of Structured Programming (1970s):

- **Structured Approach:** Edsger Dijkstra and others introduced structured programming techniques to manage complexity and improve code quality.
- **Modularization:** Breaking down programs into smaller, manageable modules became a key practice.

5. Software Engineering (1980s):

- **Formal Methods:** The adoption of formal methods for software development aimed to improve reliability and reduce errors.

- **Waterfall Model:** The waterfall model, a sequential approach to software development, gained popularity.
- 6. **Object-Oriented Programming (1980s-1990s):**
 - **OOP Paradigm:** Object-oriented programming, with languages like C++ and later Java, introduced concepts like encapsulation and inheritance.
 - **Reusability:** OOP promoted code reusability through the creation of objects and classes.
- 7. **Internet Boom (1990s):**
 - **Web Development:** The rise of the internet brought about a surge in web development. HTML, JavaScript, and CSS became essential technologies.
 - **E-commerce and Connectivity:** Challenges included building scalable systems for e-commerce and connecting diverse technologies.
- 8. **Agile Movement (2000s):**
 - **Agile Manifesto:** Agile methodologies emerged, emphasizing iterative development, collaboration, and adaptability.
 - **Customer-Centric:** The focus shifted to delivering value to customers in shorter cycles.
- 9. **Mobile Revolution (2010s):**
 - **Mobile App Development:** The proliferation of smartphones led to a boom in mobile app development.
 - **Security Concerns:** With the increase in connectivity, security became a major concern, leading to a focus on secure coding practices.
- 10. **Cloud Computing (2010s):**
 - **Scalability:** Cloud platforms provided scalable infrastructure, changing the way applications were developed and deployed.
 - **Microservices:** A shift towards microservices architecture for building large, complex applications.
- 11. **Machine Learning and AI (2010s-Present):**
 - **Integration of AI:** Machine learning and artificial intelligence became integral to various applications.
 - **Ethical Concerns:** Developers faced ethical challenges related to the use of AI, including bias in algorithms.
- 12. **DevOps and Continuous Delivery (2010s-Present):**
 - **Automation:** DevOps practices emerged, emphasizing collaboration between development and operations teams and the automation of the software delivery process.
 - **Continuous Integration/Continuous Deployment (CI/CD):** Rapid and automated deployment of software became a standard practice.
- 13. **Current Challenges (2020s):**
 - **Security Threats:** Cybersecurity remains a significant challenge, with the increasing sophistication of cyber threats.
 - **Complexity:** Building and maintaining complex, interconnected systems continue to pose challenges.
 - **Diversity and Inclusion:** The tech industry faces ongoing challenges related to diversity and inclusion.

Need for Systematic process

Addressing issues

The need for a systematic process in addressing issues and developing products is crucial for ensuring efficiency, quality, and successful outcomes. A well-defined and structured approach helps organizations and development teams navigate complexities, minimize risks, and deliver products that meet or exceed user expectations. Here are some key reasons why a systematic process is essential:

1. **Consistency and Predictability:**
 - A systematic process provides a consistent framework for development, making it easier to predict outcomes and manage expectations. This is crucial for project planning, resource allocation, and meeting deadlines.
2. **Quality Assurance:**
 - Following a systematic process enables the implementation of quality assurance measures at every stage. This includes code reviews, testing procedures, and validation processes, ensuring that the final product is of high quality and free from critical issues.
3. **Efficiency and Productivity:**
 - A well-defined process helps in streamlining workflows, reducing redundancies, and improving overall efficiency. Developers can work more productively when they have clear guidelines and procedures to follow.
4. **Risk Mitigation:**
 - Systematic processes allow for the identification and mitigation of risks early in the development lifecycle. By conducting risk assessments and implementing risk management strategies, teams can address potential issues before they escalate.
5. **Effective Collaboration:**
 - A systematic process fosters effective collaboration among team members and different departments. Clear communication channels, defined roles, and collaborative tools contribute to smoother workflows and fewer misunderstandings.
6. **Scalability:**
 - As projects grow in complexity or additional features are added, a systematic process provides a foundation for scalability. It becomes easier to adapt and expand the development process to accommodate changing requirements.
7. **Customer Satisfaction:**
 - Following a systematic approach ensures that customer needs and expectations are consistently considered throughout the development process. This customer-centric focus contributes to higher satisfaction levels and better adoption of the final product.
8. **Adaptability to Change:**
 - An effective process is not rigid but allows for flexibility to adapt to changing requirements or unexpected challenges. Agile methodologies, for example, emphasize iterative development and continuous adaptation to evolving project needs.
9. **Documentation and Knowledge Transfer:**
 - A systematic process encourages the documentation of key decisions, code, and project-related information. This documentation is valuable for knowledge

transfer, onboarding new team members, and maintaining continuity in case of personnel changes.

10. Regulatory Compliance:

- In certain industries, adherence to regulatory standards and compliance requirements is crucial. A systematic process helps ensure that development practices align with relevant regulations, avoiding legal and financial consequences.

11. Cost Control:

- By identifying and addressing issues early in the development process, a systematic approach helps control costs associated with rework, bug fixes, and late-stage modifications.

Products

A systematic process for developing, producing, and managing products is essential for various reasons. Whether it's in the context of manufacturing, software development, or any other product-oriented industry, adopting a structured and systematic approach offers several advantages:

1. **Quality Assurance:** A systematic process helps ensure the quality of the product. By defining and following a set of standards and procedures, organizations can consistently produce products that meet or exceed customer expectations.
2. **Efficiency and Cost Reduction:** A well-defined process streamlines production workflows, reduces waste, and optimizes resource utilization. This efficiency not only lowers production costs but also enhances the overall competitiveness of the product in the market.
3. **Consistency in Output:** A systematic approach ensures consistency in the final product. This is crucial for building a brand reputation and customer trust. Consumers often expect reliability and uniformity in the products they purchase.
4. **Risk Management:** By identifying potential risks and incorporating risk management strategies into the product development process, organizations can mitigate the impact of unforeseen challenges. This includes risks related to technology, supply chain, market changes, and more.
5. **Timeline and Deadline Management:** A systematic process helps in setting realistic timelines and managing deadlines effectively. This is critical for delivering products to the market on time, staying competitive, and meeting customer demands.
6. **Customer Satisfaction:** A systematic approach focuses on understanding customer needs and expectations. By incorporating customer feedback into the development process, organizations can create products that better satisfy market demands, leading to increased customer satisfaction.
7. **Innovation and Continuous Improvement:** A systematic process allows organizations to foster innovation and continuous improvement. By regularly reviewing and updating processes, companies can adapt to changing market conditions, incorporate new technologies, and stay ahead of the competition.
8. **Regulatory Compliance:** In many industries, there are specific regulations and standards that products must meet. A systematic process ensures that products adhere to these regulatory requirements, reducing the risk of legal issues and ensuring the safety and reliability of the product.

9. **Resource Optimization:** Through systematic planning and resource allocation, organizations can optimize the use of materials, labor, and other resources. This contributes to sustainability efforts and minimizes environmental impact.
10. **Collaboration and Communication:** A systematic product development process facilitates collaboration among cross-functional teams. Clear communication channels and defined roles help teams work together cohesively, reducing the likelihood of errors and misunderstandings.
11. **Documentation and Knowledge Management:** Systematic processes encourage proper documentation of each stage in the product development lifecycle. This documentation is valuable for troubleshooting, knowledge transfer, and future improvements.
12. **Market Adaptability:** A systematic approach allows organizations to adapt quickly to changes in the market, technological advancements, and evolving customer preferences. This adaptability is crucial for long-term success.

Custom solutions

Implementing a systematic process for developing custom solutions, such as software applications or tailored services, is critical for several reasons. Here are some key considerations:

1. **Client Requirements:** Custom solutions are built to address specific client needs. A systematic process helps in thoroughly understanding and documenting these requirements, ensuring that the final solution aligns with the client's expectations.
2. **Quality Assurance:** Systematic processes include quality assurance measures throughout the development lifecycle. This helps identify and rectify issues early in the process, ensuring that the custom solution meets high-quality standards and performs reliably.
3. **Efficiency and Productivity:** A systematic approach streamlines development workflows, promoting efficiency and productivity. This is particularly important in custom solutions where each project may have unique requirements. An organized process helps teams manage tasks, timelines, and resources effectively.
4. **Risk Management:** Identifying and managing risks is crucial in custom solution development. A systematic process includes risk assessment and mitigation strategies to address potential challenges, reducing the likelihood of project delays or failures.
5. **Scalability:** A well-defined process allows for scalability, enabling teams to handle projects of varying sizes and complexities. It provides a foundation for growth and ensures that custom solutions can adapt to changing demands or increasing user loads.
6. **Communication and Collaboration:** Custom solutions often involve collaboration between different stakeholders, including clients, developers, designers, and project managers. A systematic process fosters clear communication channels, ensuring that all team members are on the same page and working towards common goals.
7. **Adaptability to Technology Changes:** In the rapidly evolving field of technology, staying up-to-date is crucial. A systematic process encourages regular evaluations of new technologies and methodologies, allowing teams to incorporate the latest advancements into custom solutions.
8. **Customer Satisfaction:** A systematic approach emphasizes client involvement throughout the development process. Regular feedback loops and demonstrations ensure that the client's vision is being realized, leading to higher customer satisfaction with the final product.

9. **Documentation:** Thorough documentation is essential for custom solutions. It serves as a reference for the development team, aids in troubleshooting and maintenance, and provides valuable insights for future projects.
10. **Project Transparency:** Clients often appreciate transparency in the development process. A systematic approach includes mechanisms for reporting progress, tracking milestones, and managing client expectations, fostering trust and confidence.
11. **Cost Control:** Systematic processes help in estimating, monitoring, and controlling project costs. This is important for both the development team and the client to ensure that the project stays within budget constraints.
12. **Legal and Compliance Considerations:** Depending on the industry and nature of the custom solution, there may be legal and compliance requirements. A systematic process ensures that these considerations are identified and addressed appropriately.

Services

A systematic process for delivering services is crucial for various reasons, providing a structured approach to planning, execution, and improvement. Here are some key considerations for the need of a systematic process for services:

1. **Consistency:** A systematic process ensures consistency in service delivery. This is critical for building trust and reliability among clients, as they expect a consistent level of quality and performance from service providers.
2. **Quality Assurance:** A systematic approach includes mechanisms for quality assurance and control throughout the service delivery lifecycle. This helps identify and rectify issues, ensuring that services meet or exceed established standards.
3. **Efficiency and Productivity:** By defining and following a systematic process, service providers can optimize workflows, allocate resources effectively, and enhance overall efficiency. This contributes to increased productivity and better utilization of time and resources.
4. **Customer Satisfaction:** Understanding and meeting customer expectations is central to successful service delivery. A systematic process allows service providers to better understand customer needs, incorporate feedback, and continuously improve to enhance customer satisfaction.
5. **Risk Management:** Identifying and managing risks is essential in service delivery. A systematic approach includes risk assessment and mitigation strategies, reducing the likelihood of service disruptions or failures.
6. **Scalability:** A well-defined process enables service providers to scale their operations to handle increased demand or changes in scope. This scalability is crucial for adapting to growth and ensuring consistent service levels.
7. **Communication and Collaboration:** Effective communication is fundamental in service delivery, especially when dealing with clients and diverse teams. A systematic process includes clear communication channels and collaboration mechanisms to ensure that all stakeholders are aligned and informed.
8. **Adaptability to Change:** Services often need to adapt to changing circumstances, such as market trends, client requirements, or technological advancements. A systematic process provides a framework for incorporating changes while maintaining service quality and efficiency.
9. **Documentation:** Thorough documentation of service processes, procedures, and interactions is essential. It serves as a reference for service providers, aids in training new staff, and ensures that best practices are followed consistently.

10. **Employee Training and Development:** A systematic process includes training programs for employees, ensuring that they have the necessary skills and knowledge to deliver services effectively. This contributes to employee satisfaction and enhances the overall competence of the workforce.
11. **Performance Measurement and Metrics:** Establishing key performance indicators (KPIs) and metrics is an integral part of a systematic process. It allows service providers to measure performance, identify areas for improvement, and demonstrate value to clients.
12. **Legal and Compliance Considerations:** Depending on the nature of the services, there may be legal and regulatory requirements. A systematic process ensures that services comply with relevant laws and standards.

Domains

When discussing the need for a systematic process for domains, it's important to clarify what is meant by "domains." In the context of your question, I'll assume you are referring to areas of expertise, disciplines, or knowledge domains. Whether it's in academia, professional fields, or any specialized area, implementing a systematic approach has several advantages:

1. **Knowledge Management:** A systematic process helps organize and manage knowledge within a domain. This includes creating taxonomies, classifying information, and establishing structures that make it easier to navigate and understand the domain.
2. **Research and Development:** In academic or research domains, a systematic process aids in conducting rigorous and replicable research. This involves defining research questions, methodologies, and analysis processes to ensure the validity and reliability of findings.
3. **Problem Solving:** A systematic approach is crucial for problem-solving within a specific domain. It provides a structured framework for identifying issues, analyzing root causes, and implementing effective solutions based on established principles and methodologies.
4. **Education and Training:** In educational domains, a systematic process supports curriculum development, lesson planning, and instructional design. It ensures that learners are guided through a structured learning experience that builds a strong foundation in the domain.
5. **Standardization:** Systematic processes contribute to the development of standards within a domain. This is particularly important in professional fields where adhering to standards ensures consistency, quality, and safety in practices and outcomes.
6. **Continuous Learning:** Domains are dynamic and evolve over time. A systematic process encourages continuous learning and adaptation to new information, technologies, and best practices within a given field.
7. **Communication:** Systematic processes facilitate effective communication within a domain. This includes the development of a common language, standardized terminology, and communication channels that enable professionals within the domain to collaborate more efficiently.
8. **Innovation:** While systematic processes provide a structured foundation, they also encourage innovation within a domain. By understanding established principles and methodologies, professionals can creatively build upon existing knowledge to push the boundaries of what is known.

9. **Quality Assurance:** Whether in research, industry, or academia, a systematic approach supports quality assurance. This involves implementing processes to ensure that work within the domain meets predetermined standards and requirements.
10. **Collaboration:** In interdisciplinary domains, a systematic approach facilitates collaboration between different areas of expertise. Establishing common ground and processes enhances teamwork and the ability to address complex challenges that span multiple domains.
11. **Regulatory Compliance:** In certain domains, adherence to regulations and standards is essential. A systematic process ensures that professionals within the domain are aware of and compliant with relevant regulations.
12. **Career Development:** Professionals within a domain can benefit from a systematic approach to career development. This includes defining career paths, identifying necessary skills and competencies, and establishing criteria for advancement within the domain.

Technologies

Implementing a systematic process for technologies is crucial for several reasons, especially in the rapidly evolving and dynamic field of technology. Here are key considerations highlighting the need for a systematic approach:

1. **Innovation Management:** A systematic process helps manage the innovation lifecycle. It allows organizations to identify emerging technologies, assess their potential impact, and systematically integrate innovations into their technology stack.
2. **Strategic Planning:** A systematic approach assists in strategic planning for technology adoption. It involves assessing business goals, aligning technology strategies with organizational objectives, and developing roadmaps for the adoption and implementation of new technologies.
3. **Efficient Development and Deployment:** Systematic processes streamline the development and deployment of technologies. This includes defining standardized development methodologies, coding standards, and deployment procedures to ensure consistency and efficiency.
4. **Risk Management:** Technology projects often come with inherent risks. A systematic process incorporates risk management strategies, helping identify potential challenges early in the project lifecycle and mitigating risks to avoid project setbacks or failures.
5. **Scalability:** As organizations grow, their technology needs evolve. A systematic approach allows for scalable technology solutions that can adapt to changing requirements, whether in terms of increased user loads, expanded functionalities, or changes in the business environment.
6. **Interoperability:** In complex technology ecosystems, interoperability is critical. A systematic process includes standards and protocols to ensure seamless integration and communication between different technologies and systems.
7. **Security and Compliance:** Cybersecurity is a top priority. A systematic process includes robust security measures and compliance checks to protect against cyber threats and ensure that technologies meet legal and regulatory requirements.
8. **Resource Optimization:** Efficient resource allocation is essential in technology projects. A systematic process helps organizations optimize the use of human resources, technology infrastructure, and budget, ensuring that resources are allocated where they are most needed.

9. **Adoption and Training:** Introducing new technologies requires user adoption and proper training. A systematic process includes change management strategies, user training programs, and documentation to facilitate smooth transitions and maximize technology adoption.
10. **Continuous Improvement:** Technology is in a constant state of evolution. A systematic approach encourages continuous improvement through regular evaluations, feedback loops, and the incorporation of lessons learned into future projects.
11. **Vendor Management:** For organizations relying on external technology vendors, a systematic process helps manage vendor relationships effectively. This includes vendor selection, contract negotiations, and ongoing performance evaluations.
12. **Data Governance:** With the increasing importance of data, a systematic approach to data governance is vital. This involves defining data management policies, ensuring data quality, and implementing processes for data security and privacy.
13. **Environmental Impact:** Consideration for the environmental impact of technology is becoming more important. A systematic process can include criteria for evaluating and minimizing the environmental footprint of technology solutions.

Software life cycle

A systematic process for the software life cycle is essential for the successful development, deployment, and maintenance of software applications. The software life cycle refers to the stages that a software product goes through from its inception to retirement. Here are key reasons highlighting the need for a systematic approach:

1. **Requirements Management:** A systematic process helps in eliciting, analyzing, and managing requirements effectively. This ensures that the software solution addresses the needs of users and stakeholders.
2. **Planning and Estimation:** Systematic processes support project planning and estimation. This involves defining project scope, allocating resources, estimating timelines, and setting realistic expectations for stakeholders.
3. **Design and Architecture:** A systematic approach facilitates the creation of a robust software design and architecture. This includes defining the system's structure, components, modules, and the relationships between them.
4. **Development Standards:** Establishing and adhering to coding standards is crucial for consistency and maintainability. A systematic process includes guidelines for writing clean, readable, and maintainable code.
5. **Quality Assurance and Testing:** Systematic processes incorporate quality assurance and testing throughout the software development life cycle. This ensures that software meets quality standards and is free of critical defects.
6. **Version Control:** A systematic process includes version control mechanisms to manage changes to the source code systematically. This helps in tracking revisions, collaborating with multiple developers, and rolling back changes if necessary.
7. **Deployment and Release Management:** Systematic processes guide the deployment and release of software products. This includes planning for deployment, ensuring a smooth transition to production, and managing version releases.
8. **Change Management:** Software evolves, and changes are inevitable. A systematic approach includes change management processes to handle modifications, enhancements, and bug fixes effectively.

9. **Documentation:** Thorough documentation is crucial for understanding the software system. Systematic processes include the creation of documentation for code, user manuals, technical specifications, and other relevant materials.
10. **Collaboration and Communication:** Effective communication and collaboration are critical for successful software development. Systematic processes define communication channels, collaboration tools, and reporting structures.
11. **User Training:** Introducing new software to end-users requires training. A systematic process includes planning and implementing user training programs to ensure users can effectively utilize the software.
12. **Maintenance and Support:** After deployment, software requires ongoing maintenance and support. A systematic process includes procedures for handling software updates, addressing issues, and providing customer support.
13. **Security:** Systematic processes address security considerations at every stage of the software life cycle. This includes secure coding practices, vulnerability assessments, and implementing measures to protect against cyber threats.
14. **Scalability and Performance:** Systematic processes consider scalability and performance requirements. This involves designing software that can handle increased workloads and optimizing performance to meet user expectations.
15. **Feedback and Continuous Improvement:** A systematic approach includes mechanisms for collecting user feedback and continuously improving the software. This iterative feedback loop helps enhance the software's features and usability.

Software development lifecycle

The Software Development Life Cycle (SDLC) is a systematic process or framework followed by software developers to design, develop, test, and deploy high-quality software. The SDLC is a series of phases that guide the development of software applications, ensuring that the final product meets user requirements and is delivered within budget and on time. While different methodologies may have variations in their phases, the core stages of the SDLC typically include:

1. **Requirements Gathering and Analysis:**
 - **Objective:** Understand the needs and expectations of end-users and stakeholders.
 - **Activities:** Collect and analyze requirements, define project scope, and create a detailed requirements document.
2. **Planning:**
 - **Objective:** Define project goals, timelines, resources, and deliverables.
 - **Activities:** Develop a project plan, allocate resources, estimate costs, and establish timelines.
3. **Design:**
 - **Objective:** Create a blueprint for the software solution based on the requirements.
 - **Activities:** Develop system architecture, design the user interface, define data structures, and create detailed technical specifications.
4. **Implementation (Coding):**
 - **Objective:** Translate the design into actual code.
 - **Activities:** Write code according to coding standards, perform unit testing, and ensure code quality.
5. **Testing:**

- **Objective:** Identify and fix defects to ensure the software meets quality standards.
 - **Activities:** Conduct various types of testing (unit testing, integration testing, system testing, and acceptance testing) to validate the software's functionality.
6. **Deployment:**
- **Objective:** Release the software to the production environment.
 - **Activities:** Prepare for deployment, migrate data, and launch the software for end-users.
7. **Maintenance and Support:**
- **Objective:** Address issues, update features, and provide ongoing support.
 - **Activities:** Monitor software performance, fix bugs, release updates, and provide user support.

These stages often follow a linear sequence, commonly known as the Waterfall model. However, many modern software development methodologies, such as Agile, Scrum, and DevOps, emphasize iterative and collaborative approaches, allowing for flexibility and adaptation throughout the SDLC. In Agile methodologies, for example, the SDLC is organized into shorter iterations or sprints, allowing for frequent reassessment and adjustments based on feedback.

The choice of the SDLC model depends on project requirements, timelines, and the development team's preferences. Each model has its strengths and weaknesses, and organizations often tailor their approach based on the specific needs of a project.

Software release process

The software release process is a systematic set of steps and activities that are followed to prepare, test, and deploy a new version or update of software to end-users. The goal of the release process is to ensure that the software is of high quality, meets user expectations, and is successfully delivered to users with minimal disruptions. While specific steps may vary based on the organization and the type of software being released, the following is a general outline of key activities in a typical software release process:

1. **Release Planning:**
 - **Objective:** Define the scope and goals of the release.
 - **Activities:**
 - Identify features and changes to be included in the release.
 - Determine release timelines and deadlines.
 - Allocate resources and assign responsibilities.
2. **Code Freeze:**
 - **Objective:** Stabilize the codebase for testing.
 - **Activities:**
 - Declare a code freeze to stop the introduction of new features or changes.
 - Focus on fixing bugs and addressing critical issues.
3. **Feature Complete:**
 - **Objective:** Ensure all planned features for the release are implemented.
 - **Activities:**
 - Confirm that all intended features have been developed and integrated.
4. **Code Review:**

- **Objective:** Ensure code quality and identify potential issues.
 - **Activities:**
 - Conduct code reviews to catch bugs, enforce coding standards, and improve code quality.
 - Address feedback from code reviews.
5. **Testing:**
- **Objective:** Validate the software for quality assurance.
 - **Activities:**
 - Perform various testing types (unit testing, integration testing, system testing, and acceptance testing).
 - Fix and retest any identified issues.
6. **User Acceptance Testing (UAT):**
- **Objective:** Validate the software against user expectations.
 - **Activities:**
 - Conduct UAT with real users or stakeholders.
 - Address any feedback or issues identified during UAT.
7. **Release Candidate:**
- **Objective:** Declare a build as a release candidate if it passes testing.
 - **Activities:**
 - Stabilize the release candidate, addressing any last-minute issues.
 - Generate release documentation.
8. **Approval and Sign-off:**
- **Objective:** Obtain approval from stakeholders to proceed with the release.
 - **Activities:**
 - Present the release candidate to key stakeholders for review.
 - Obtain sign-off indicating approval for the release.
9. **Deployment:**
- **Objective:** Distribute the release to end-users.
 - **Activities:**
 - Deploy the software to production or the target environment.
 - Monitor the deployment for any issues.
10. **Post-Release Activities:**
- **Objective:** Ensure a smooth transition and address any post-release issues.
 - **Activities:**
 - Provide support to end-users and address any immediate issues.
 - Capture and analyze post-release metrics and feedback.
11. **Documentation and Communication:**
- **Objective:** Document the release and communicate changes.
 - **Activities:**
 - Update release notes and documentation.
 - Communicate the release to end-users, support teams, and other stakeholders.
12. **Continuous Improvement:**
- **Objective:** Learn from the release process for future improvements.
 - **Activities:**
 - Conduct a retrospective to review the release process.
 - Identify areas for improvement and implement changes in future releases.

Source control

Source control, also known as version control or revision control, is a critical component in software development that manages changes to source code over time. It provides a systematic way to track, manage, and coordinate changes made by multiple contributors to a codebase. The primary objectives of source control include version tracking, collaboration, and the ability to revert to previous states of the code. Here are key concepts and components associated with source control:

1. Repository:

- A repository is a central storage location that holds the source code, project files, and version history. There are two main types of repositories: centralized (where there's a single central server) and distributed (where each user has a complete copy of the repository).

2. Version:

- A version represents a specific state of the source code at a given point in time. Each change to the codebase is associated with a new version.

3. Commit:

- A commit is a record of changes made to the source code. It includes a set of modifications (additions, deletions, and modifications of files), a commit message explaining the changes, and a unique identifier.

4. Branch:

- A branch is a separate line of development within the codebase. Branches allow developers to work on features or fixes independently without affecting the main codebase until changes are ready to be merged.

5. Merge:

- Merging is the process of combining changes from one branch (e.g., a feature branch) into another (e.g., the main branch). It integrates the changes made in different branches into a unified codebase.

6. Checkout:

- Checking out refers to the act of switching between different versions or branches of the code. Developers can switch to a specific commit or branch to work on different parts of the codebase.

7. Conflict:

- A conflict occurs when two or more contributors make changes to the same part of a file concurrently. Resolving conflicts involves choosing how to combine or discard conflicting changes.

8. Tag:

- A tag is a snapshot of the codebase at a specific point in time. Tags are often used to mark releases or significant milestones and provide a stable reference for future use.

9. Remote Repository:

- A remote repository is a copy of the codebase stored on a server. Developers can push changes to the remote repository and pull changes made by others.

10. Clone:

- Cloning involves creating a copy of a repository, usually from a remote repository to a local machine. Cloning allows developers to work on their own local copies of the codebase.

11. Pull Request (PR):

- In distributed version control systems, like Git, a pull request is a proposed set of changes submitted by a developer to the repository owner. It allows for code review before merging changes into the main branch.
12. **Continuous Integration (CI):**
- CI is a practice that involves automatically testing and integrating code changes as soon as they are committed to the version control system. This helps identify and address issues early in the development process.

Versioning

Versioning in software development refers to the practice of assigning unique identifiers or labels to different versions of a software product or its components. This systematic approach allows developers, users, and other stakeholders to track and manage changes to the software over time. There are various types of versioning, including:

1. **Semantic Versioning (SemVer):**
 - Semantic Versioning is a versioning scheme that assigns version numbers based on three components: MAJOR.MINOR.PATCH. Each component has a specific meaning:
 - **MAJOR:** Increments for incompatible API changes.
 - **MINOR:** Increments for backward-compatible feature additions.
 - **PATCH:** Increments for backward-compatible bug fixes.
2. **Incremental Versioning:**
 - In incremental versioning, version numbers are incremented sequentially for each release, usually starting from 1.0. It doesn't provide specific information about the nature of changes.
3. **Date Versioning:**
 - Date versioning uses the release date as the version number. For example, a software version might be represented as "YYYY.MM.DD" (Year.Month.Day). This approach indicates the release date but doesn't convey information about the changes made.
4. **Alphanumeric Versioning:**
 - Alphanumeric versioning uses a combination of letters and numbers to represent versions. This method is less standardized and may include project-specific identifiers, release names, or other alphanumeric sequences.
5. **Build Number Versioning:**
 - Build number versioning assigns a unique identifier to each build of the software, often using a sequential number or a combination of numbers and letters. It helps track individual builds within a specific version.
6. **CalVer (Calendar Versioning):**
 - Calendar versioning uses a date-based approach similar to semantic versioning but in a more flexible manner. It may include additional information such as the year and month of release but doesn't strictly adhere to SemVer rules.
7. **Git Commit Hash Versioning:**
 - Some projects use the unique identifier of the Git commit hash as the version number. This ensures that each version corresponds to a specific state of the source code in the version control system.
8. **Major/Minor Versioning:**

- Major/Minor versioning simplifies version numbers to major and minor components, without a separate patch version. This is suitable for projects that may not have a significant number of patches or bug fixes.

Maintenance of software

Software maintenance is the process of modifying and updating a software system to correct faults, improve performance, adapt to changes in the environment, and add new features. Maintenance is a crucial phase in the software development life cycle, as it ensures that the software continues to meet its objectives and remains effective throughout its lifecycle. There are typically three types of software maintenance:

1. **Corrective Maintenance:**
 - **Objective:** Address and fix software defects or bugs.
 - **Activities:**
 - Identify and analyze reported issues.
 - Develop and test fixes for identified problems.
 - Deploy patches or updates to resolve issues.
2. **Adaptive Maintenance:**
 - **Objective:** Adapt the software to changes in its operating environment or external dependencies.
 - **Activities:**
 - Update the software to work with new hardware or software platforms.
 - Make modifications to comply with changes in regulations or standards.
 - Adapt the software to interface with new third-party systems.
3. **Perfective Maintenance:**
 - **Objective:** Enhance the software by adding new features or improving existing ones.
 - **Activities:**
 - Identify and prioritize new feature requests or improvements.
 - Design and implement new features.
 - Test and deploy updates that introduce enhancements.

Here are key considerations and best practices for the maintenance of software:

1. **Establish a Maintenance Plan:**
 - Develop a comprehensive maintenance plan that outlines the processes, roles, responsibilities, and schedules for maintenance activities.
2. **Bug Tracking and Resolution:**
 - Implement a robust bug tracking system to capture and prioritize reported issues. Ensure timely resolution of critical bugs.
3. **Version Control and Release Management:**
 - Use version control systems to manage software versions and releases systematically. Clearly document and communicate release notes for each update.
4. **Monitoring and Performance Tuning:**
 - Implement monitoring tools to detect performance issues or system failures. Regularly analyze performance metrics and optimize code or configurations as needed.
5. **Documentation:**

- Maintain up-to-date documentation, including user manuals, technical documentation, and system architecture documentation. This aids in troubleshooting and onboarding new team members.
- 6. **Security Updates:**
 - Stay vigilant about security vulnerabilities. Apply security patches promptly and conduct regular security audits to identify and address potential risks.
- 7. **User Support:**
 - Provide user support through helpdesk services or online forums. Address user inquiries, provide guidance, and offer assistance in resolving issues.
- 8. **Regression Testing:**
 - Perform regression testing after each update to ensure that new changes do not introduce unintended side effects or break existing functionality.
- 9. **Automated Testing:**
 - Invest in automated testing tools and frameworks to streamline the testing process and catch issues early in the development pipeline.
- 10. **Collaboration with Stakeholders:**
 - Maintain open communication with stakeholders, including end-users, to gather feedback, understand their needs, and prioritize maintenance activities accordingly.
- 11. **Capacity Planning:**
 - Regularly assess the system's capacity and scalability. Plan for infrastructure upgrades or optimizations to accommodate growth and increased demand.
- 12. **Continuous Improvement:**
 - Conduct post-implementation reviews after each maintenance activity. Identify areas for improvement and implement changes to enhance the efficiency of future maintenance efforts.

DevOps

DevOps, short for Development and Operations, is a set of practices and cultural philosophies that aim to enhance collaboration and communication between software development (Dev) and IT operations (Ops) teams. The primary goal of DevOps is to streamline the software delivery process, improve deployment frequency, achieve faster time-to-market, and ensure a more reliable and efficient software development life cycle. DevOps is often seen as a cultural shift, emphasizing collaboration, automation, and continuous improvement.

Key principles and practices of DevOps include:

1. **Collaboration:**
 - Foster a culture of collaboration and shared responsibility between development and operations teams. Break down silos and encourage open communication to enhance understanding and cooperation.
2. **Automation:**
 - Automate repetitive and manual tasks throughout the software development life cycle, including building, testing, deployment, and monitoring. Automation improves efficiency, reduces errors, and accelerates the release process.
3. **Continuous Integration (CI):**
 - Integrate code changes frequently, typically multiple times a day, by automatically triggering builds and running automated tests. CI helps identify and address integration issues early in the development process.

4. **Continuous Deployment (CD):**
 - Automate the deployment process to release software updates continuously to production after passing automated tests. Continuous Deployment ensures that code changes are rapidly and reliably delivered to end-users.
5. **Infrastructure as Code (IaC):**
 - Manage and provision infrastructure using code. IaC allows teams to automate the configuration and deployment of infrastructure, reducing manual errors and ensuring consistency across environments.
6. **Monitoring and Logging:**
 - Implement robust monitoring and logging practices to gain insights into the performance, availability, and health of applications and infrastructure. Proactive monitoring helps identify and address issues before they impact users.
7. **Microservices Architecture:**
 - Adopt a microservices architecture, breaking down applications into smaller, independently deployable services. Microservices promote agility, scalability, and ease of maintenance.
8. **Containerization:**
 - Use containerization technologies like Docker to package applications and their dependencies into containers. Containers provide consistency across different environments and simplify deployment.
9. **Orchestration:**
 - Use container orchestration tools, such as Kubernetes, to automate the deployment, scaling, and management of containerized applications. Orchestration ensures efficient resource utilization and high availability.
10. **Continuous Feedback:**
 - Establish feedback loops throughout the development process. Gather feedback from users, operations, and automated testing to identify areas for improvement and iterate on development practices.
11. **Security Integration (DevSecOps):**
 - Integrate security practices throughout the DevOps pipeline. Embed security checks into the development and deployment processes to identify and address security vulnerabilities early.
12. **Culture of Continuous Learning:**
 - Encourage a culture of continuous learning and improvement. Embrace feedback, learn from failures, and iterate on processes to enhance efficiency and effectiveness.

Software Development Processes

A PROCESS FRAMEWORK

- ✓ Establishes the foundation for a complete software process
- ✓ Identifies a number of framework activities applicable to all software projects
- ✓ Also include a set of umbrella activities that are applicable across the entire software process.

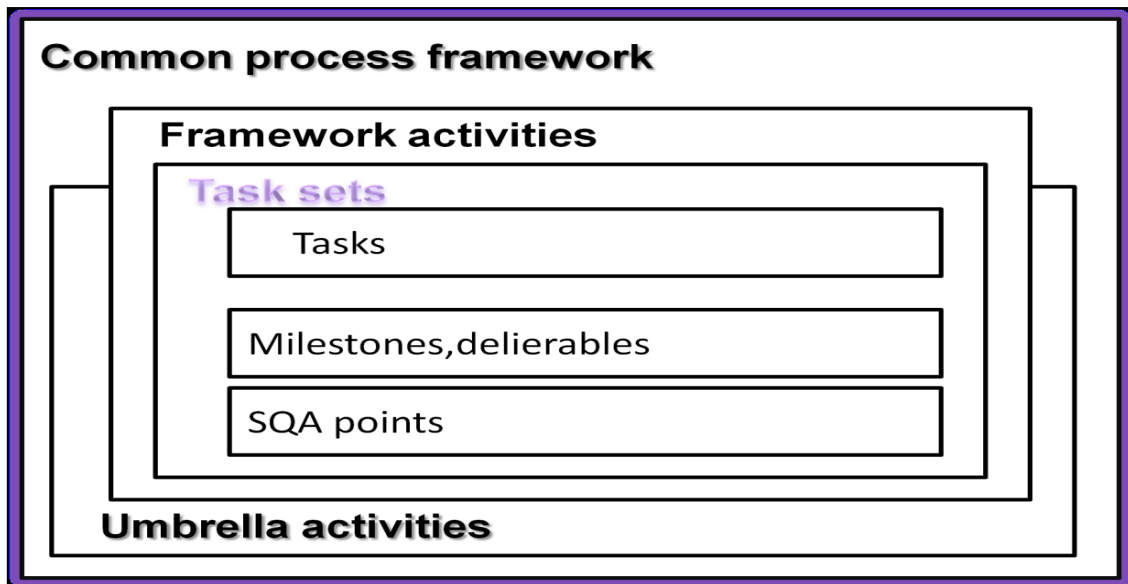


Figure: The Process Framework

- ✓ Used as a basis for the description of process models.
- ✓ Generic process activities
 - Communication
 - Planning
 - Modeling
 - Construction
 - Deployment
- ✓ Generic view of engineering complimented by a number of umbrella activities
 - Software project tracking and control
 - Risk management
 - Software quality assurance
 - Formal technical reviews
 - Measurement
 - Software configuration management
 - Reusability management
 - Document preparation and production
 - Risk management

Generic Process Model

- A **process** is a collection of activities, actions and tasks that are performed when some work product is to be created. Purpose of process is to deliver software in a timely manner and with sufficient quality to satisfy those who have sponsored its creation and those who will use it.

- An **activity** strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.
- An **action** (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural model).
- A **task** focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

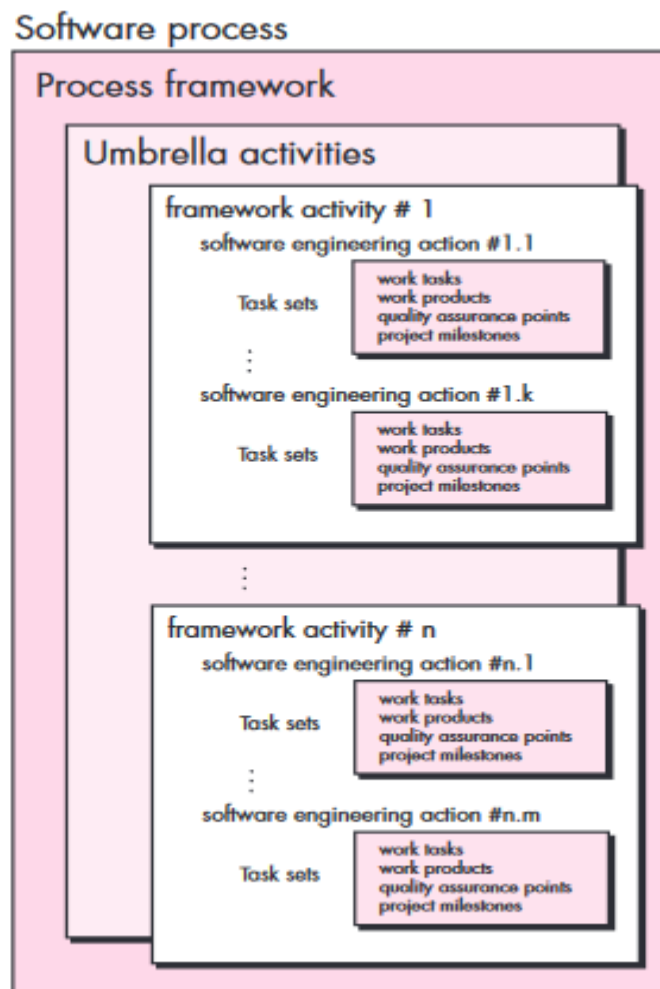


Figure: A software Process Framework

Process flow—describes how the framework activities and the actions and tasks that occur within each framework activity are organized concerning sequence and time.

Types of Process Flows:

- ✓ A linear process flow executes each of the five framework activities in sequence, beginning with communication and culminating with deployment

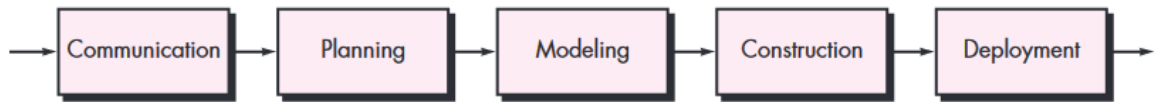


Fig: Linear Process Flow

- ✓ An iterative process flow repeats one or more of the activities before proceeding to the next.

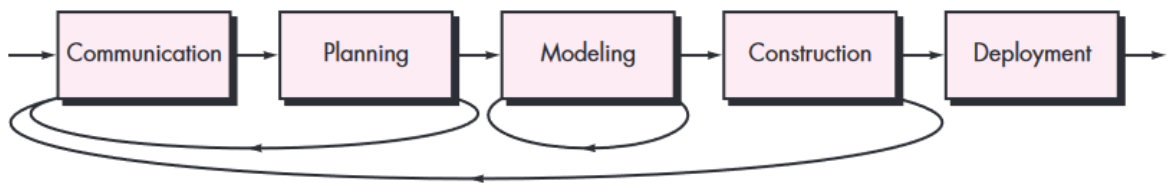


Fig: Iterative Process Flow

- ✓ An evolutionary process flow executes the activities in a “circular” manner. Each circuit through the five activities leads to a more complete version of the software.

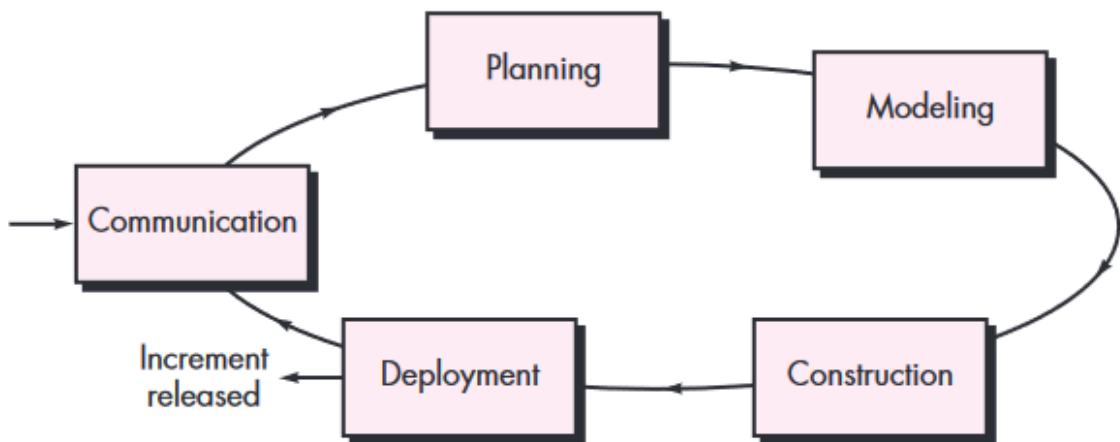


Fig: Evolutionary Process Flow

- ✓ A parallel process flow executes one or more activities in parallel with other activities (e.g., modeling for one aspect of the software might be executed in parallel with the construction of another aspect of the software).

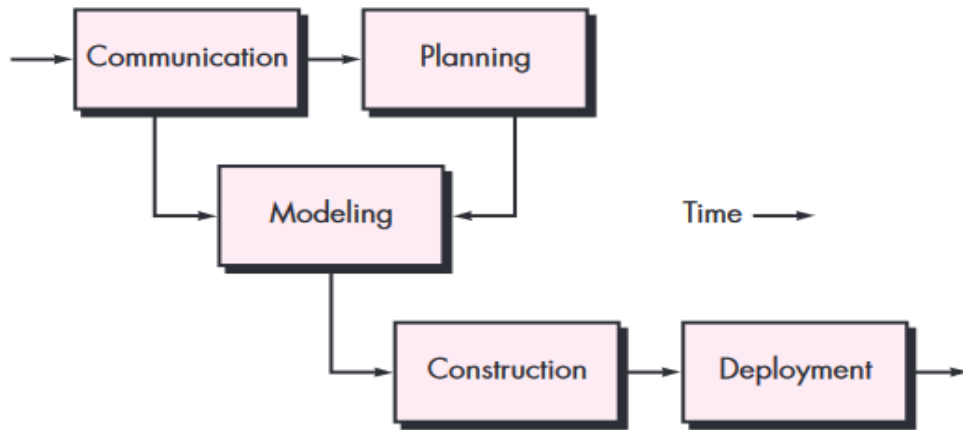


Fig: Parallel Process Flow

Perspective Process Models:

Various software development life cycle [SDLC] models are suitable for specific project-related conditions which include organization, requirements stability, risks, budget, and duration of the project. One life cycle model theoretical may suit particular conditions and at the same time other models may also look fit the requirements but one should consider the trade-off while deciding which model to choose. Here I am summarizing the advantages and disadvantages of various life cycle models.

- ✓ Help in software development.
- ✓ Guide the software team through a set of framework activities
- ✓ Process Models may be linear, incremental, or evolutionary.

The Classic Model / The Linear Model / the Waterfall Model

- ✓ Used when requirements are well understood in the beginning.
- ✓ Also called the classic life cycle.
- ✓ A systematic, sequential approach to Software development.
- ✓ Begins with customer specification of Requirements and progresses through planning, modeling, construction, and deployment.
- ✓ This Model suggests a systematic, sequential approach to SW development that begins at the system level and progresses through analysis, design, code, and testing.

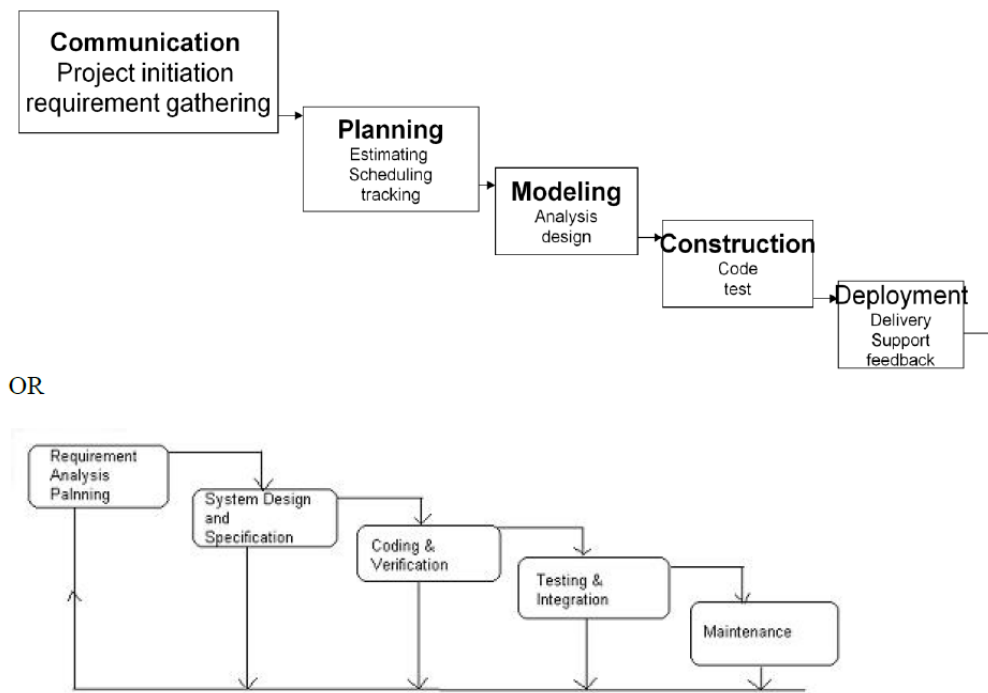


Figure: The Waterfall Model.

Advantages

- ✓ Simple goal.
- ✓ Simple to understand and use.
- ✓ Clearly defined stages.
- ✓ Well understood milestones.
- ✓ Easy to arrange tasks.
- ✓ Process and results are well documented.
- ✓ Easy to manage. Each phase has a specific deliverable and a review.
- ✓ Works well for projects where requirements are well understood.
- ✓ Works well when quality is more important than cost/schedule.
- ✓ Customers/End users already know about it.

Disadvantages

- ✓ It is difficult to measure progress within stages.
- ✓ Cannot accommodate changing requirements.
- ✓ No working software is produced until late in the life cycle.
- ✓ Risk and uncertainty are high with this process model.
- ✓ Adjusting scope during the life cycle can end a project
- ✓ Not suitable for complex projects

- ✓ Not suitable for projects of long duration because in long-running projects requirements are likely to change.
- ✓ Integration is done as a "big-bang" at the very end [Single shot], which doesn't allow identifying any technological or business bottleneck or challenges early.
- ✓ Users can only judge quality at the end.
- ✓ Attempting to go back 2 or more phases is very costly.
- ✓ Percentage completion of functionality cannot be determined in the middle of the project development because functionality will be undergoing some phase.
- ✓ Very risky, since one process can not start before finishing the other.

Problems

- ✓ Real projects rarely follow the sequential flow since they are always iterative
- ✓ The model requires requirements to be explicitly spelled out at the beginning, which is often difficult.
- ✓ A working model is not available until late in the project time plan.
- ✓ The customer must have patience.

V-Model

- ✓ A variation in the representation of the waterfall model is called the *V-model*.
- ✓ The V-model depicts the relationship of quality assurance actions to the actions associated with communication, modeling, and early construction activities.
- ✓ As the software team moves down the left side of the V, basic problem requirements are refined into progressively more detailed and technical representations of the problem and its solution.
- ✓ Once code has been generated, the team moves up the right side of the V, essentially performing a series of tests (quality assurance actions) that validate each of the models created as the team moved down the left side.
- ✓ There is no fundamental difference between the classic life cycle and the V-model. The V-model provides a way of visualizing how verification and validation actions are applied to earlier engineering work.

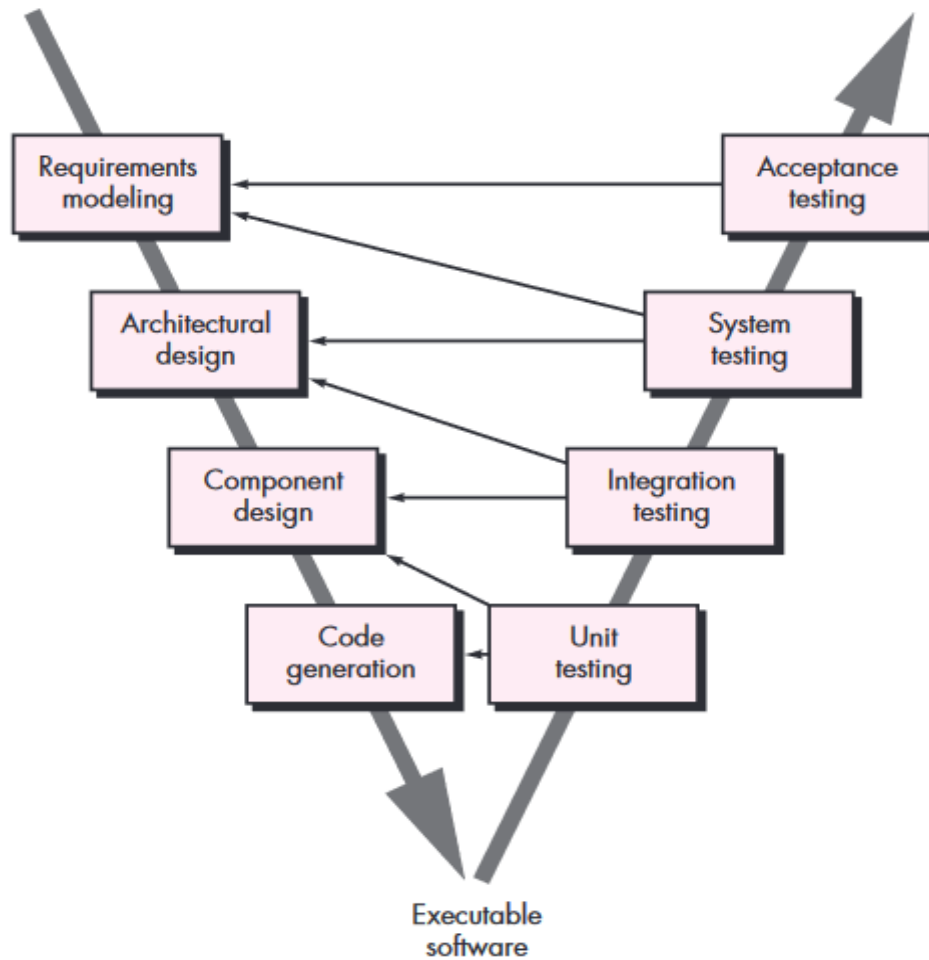


Figure: The V Model

The Incremental Process Model

- ✓ Linear sequential model is not suited for iterative projects, Incremental model suits such projects.
- ✓ Used when initial requirements are reasonably well-defined and compelling needs to provide limited functionality quickly.
- ✓ Functionality expanded further in later releases.
- ✓ Software is developed in increments.
- ✓ Software releases in increments.
- ✓ 1st increment constitutes the Core product.
- ✓ Basic requirements are addressed.
- ✓ Core product undergoes detailed evaluation by the customer.
- ✓ As a result, a plan is developed for the next increment.

- ✓ Plan addresses the modification of core products to better meet the needs of the customer.
- ✓ Process is repeated until the complete product is produced.

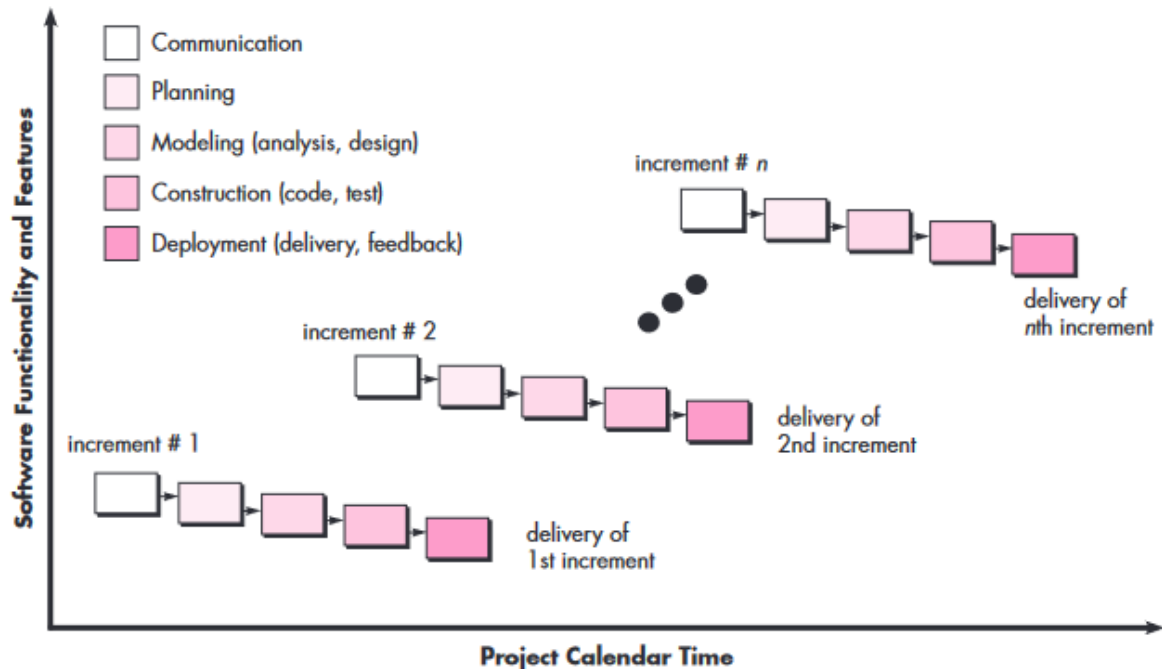


Figure: The incremental Model

Advantages

- ✓ Some working functionality can be developed quickly and early in the life cycle.
- ✓ Results are obtained early and periodically.
- ✓ Parallel development can be planned.
- ✓ Progress can be measured.
- ✓ Less costly to change the scope/requirements.
- ✓ Testing and debugging during smaller iterations are easy.
- ✓ Risks are identified and resolved during iteration, and each iteration is an easily managed milestone.
- ✓ Easier to manage risk – High-risk part is done first.
- ✓ With every increment operational product is delivered.
- ✓ Issues, challenges & risks identified from each increment can be utilized/applied to the next increment.

Disadvantages

- ✓ More resources may be required.
- ✓ Although the cost of change is lesser it is not very suitable for changing requirements.
- ✓ More management attention is required.
- ✓ Each phase of iteration is rigid with no overlaps.
- ✓ System architecture or design issues may arise because not all requirements are gathered upfront for the entire life cycle.
- ✓ Does not allow iterations within an increment.
- ✓ Defining increments may require the definition of the complete system.

The RAD Model (Rapid Application Development)

- ✓ An incremental software process model.
- ✓ Having a short development cycle.
- ✓ High-speed adoption of the waterfall model using a component-based construction approach.
- ✓ Creates a fully functional system within a very short span time of 60 to 90 days.
- ✓ Multiple software teams work in parallel on different functions.
- ✓ Modeling encompasses three major phases: Business modeling, Data modeling, and process modeling.
- ✓ Construction uses reusable components, automatic code generation, and testing.

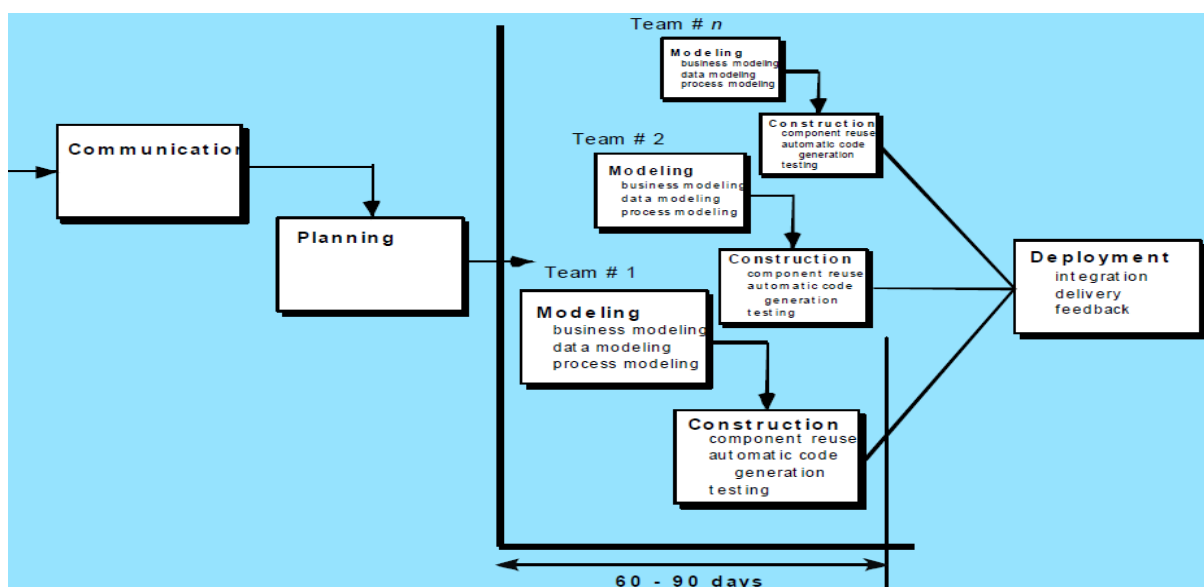


Figure: The RAD Model

Advantages

- ✓ Time to deliver is less.
- ✓ Changing requirements can be accommodated.
- ✓ Progress can be measured.
- ✓ Cycle time can be short with the use of powerful RAD tools.
- ✓ Productivity with fewer people in a short time.
- ✓ Use of tools and frameworks.

Disadvantages

- ✓ Management complexity is more.
- ✓ Resource requirements may be more.
- ✓ Suitable for systems that are component-based and scalable.
- ✓ Suitable only when requirements are well known.
- ✓ Requires user involvement throughout the life cycle.
- ✓ Suitable for a project requiring shorter development times.

Problems in RAD

- ✓ Requires a number of RAD teams
- ✓ Requires commitment from both developer and customer for rapid-fire completion of activities.
- ✓ Requires modularity.
- ✓ Not suited when technical risks are high.
- ✓ For large, but scalable projects, RAD requires sufficient human resources to create the right number of RAD teams.
- ✓ If developers and customers are not committed to the rapid-fire activities necessary, RAD projects will fail.
- ✓ If a system cannot be properly modularized, building the components necessary for RAD will be problematic.
- ✓ RAD may not be appropriate when technical risks are high.

Evolutionary Process Models

- ✓ Software evolves over a period.
- ✓ Business and product requirements often change as development proceeds making a straight-line path to an end product unrealistic
- ✓ Evolutionary models are iterative and as such apply to modern-day applications
- ✓ Types of evolutionary models
 - Prototyping
 - Spiral model

Prototyping

The Prototyping Model is a systems development method (SDM) in which a prototype (an early approximation of a final system or product) is built, tested, and then reworked as necessary until an acceptable prototype is finally achieved from which the complete system or product can now be developed. This model works best in scenarios where not all the project requirements are known in detail ahead of time. It is an iterative, trial-and-error process that takes place between the developers and the users.

- ✓ Mockup or model (throw away version) of a software product.
- ✓ Used when the customer defines a set of objectives but does not identify input, output, or processing requirements.
- ✓ Developer is not sure of:
 - The efficiency of an algorithm.
 - The adaptability of an operating system.
 - Human/machine interaction.

Steps in Prototyping

- ✓ The new system requirements are defined in as much detail as possible. This usually involves interviewing a number of users representing all the departments or aspects of the existing system.
- ✓ A preliminary design is created for the new system.
- ✓ A prototype of the new system is constructed from the preliminary design. This is usually a scaled-down system and represents an approximation of the characteristics of the final product.

- ✓ The users thoroughly evaluate the prototype, noting its strengths and weaknesses, what needs to be added, and what should be removed. The developer collects and analyzes the remarks from the users.
- ✓ The prototype is modified, based on the comments supplied by the users, and a second prototype of the new system is constructed.
- ✓ The second prototype is evaluated in the same manner as the prototype.
- ✓ The preceding steps are iterated as many times as necessary until the users are satisfied that the prototype represents the final product desired.
- ✓ The final system is constructed, based on the final prototype.
- ✓ The final system is thoroughly evaluated and tested. Routine maintenance is carried out continuingly to prevent large-scale failures and minimize downtime.

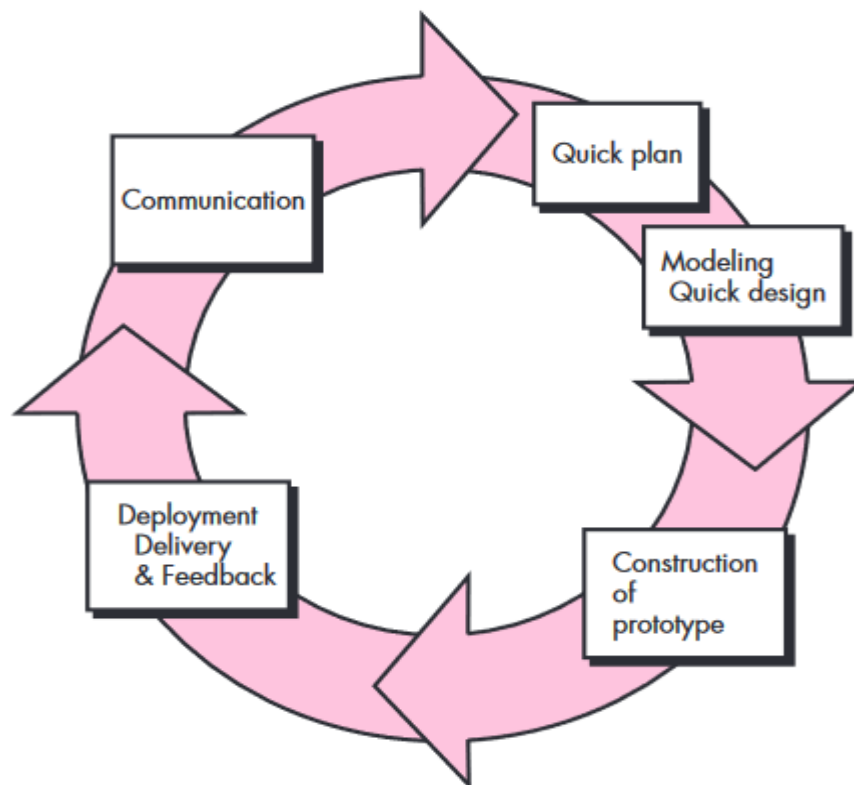


Figure: The Prototype Model

Steps in Prototyping in Short

- ✓ Begins with requirement gathering.
- ✓ Identify whatever requirements are known.
- ✓ Outline areas where the further definition is mandatory.
- ✓ Quick designs occur.
- ✓ Quick design leads to the construction of a prototype.

- ✓ Prototype is evaluated by the customer.
- ✓ Requirements are refined.
- ✓ Prototype is turned to satisfy the needs of the customer.

Advantages

- ✓ Reduced time and costs.
- ✓ Improved and increased user involvement.

Disadvantages

- ✓ Insufficient analysis.
- ✓ User confusion of prototype and finished system.
- ✓ Developer misunderstanding of user objectives.
- ✓ Developer attachment to prototype.
- ✓ Excessive development time of the prototype.
- ✓ Expense of implementing prototyping.

Limitation of Prototyping

- ✓ In a rush to get it working, overall software quality or long-term maintainability are generally overlooked.
- ✓ Use of inappropriate OS or PL.
- ✓ Use of the inefficient algorithm.

The Spiral Model

- ✓ An evolutionary model which combines the best feature of the classical life cycle and the iterative nature of the prototype model
- ✓ Include new element: Risk element
- ✓ Starts in the middle and continually visits the basic tasks of communication, planning, modeling, construction, and deployment
- ✓ Realistic approach to the development of large-scale systems and software
- ✓ Software evolves as the process progresses
- ✓ Better understanding between developer and customer
- ✓ The first circuit might result in the development of a product specification
- ✓ Subsequent circuits develop a prototype
- ✓ And sophisticated version of the software

Advantages

- ✓ Changing requirements can be accommodated.
- ✓ Allows for extensive use of prototypes.
- ✓ Requirements can be captured more accurately.
- ✓ Users see the system early.
- ✓ Development can be divided into smaller parts and more risky parts can be developed earlier which helps better risk management.

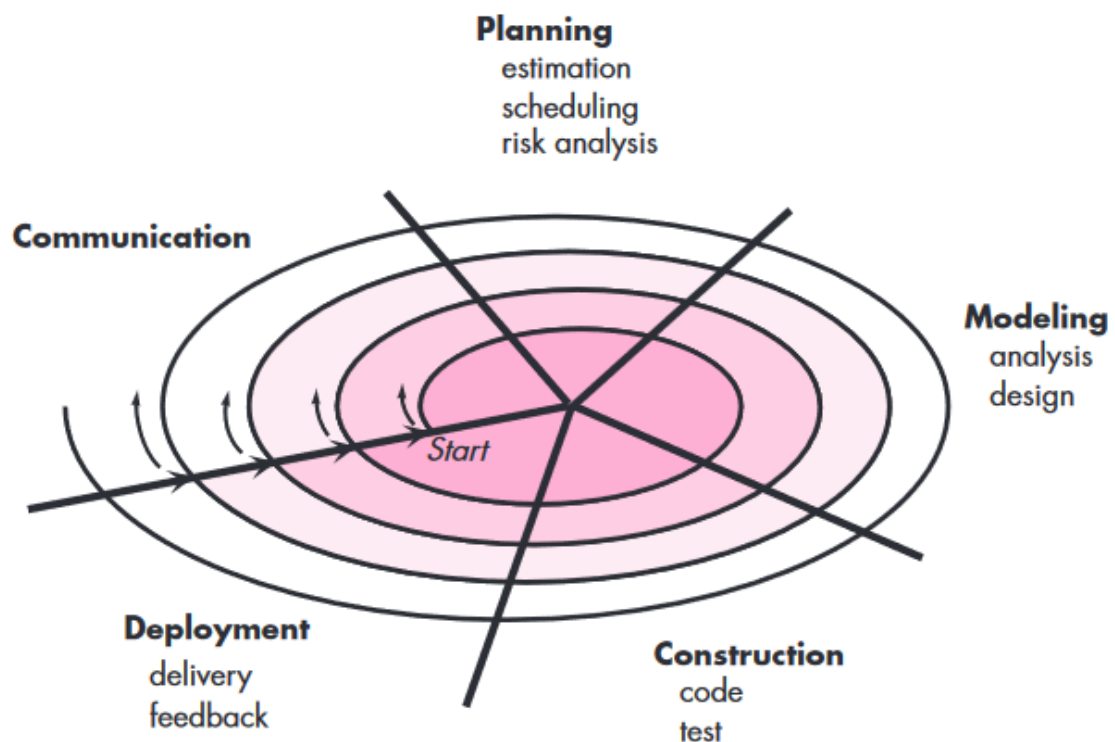


Figure: The Spiral Model

Disadvantages

- ✓ Management is more complex.
- ✓ End of the project may not be known early.
- ✓ Not suitable for small or low-risk projects (expensive for small projects).
- ✓ process is complex.
- ✓ Spiral may go indefinitely.
- ✓ A Large number of intermediate stages require excessive documentation.

Problems in Evolutionary Process

- ✓ Difficult in project planning.
- ✓ Speed of evolution is not known.
- ✓ Does not focus on flexibility and extensibility (more emphasis on high quality).
- ✓ Requirement is a balance between high quality and flexibility and extensibility.

The Concurrent Model

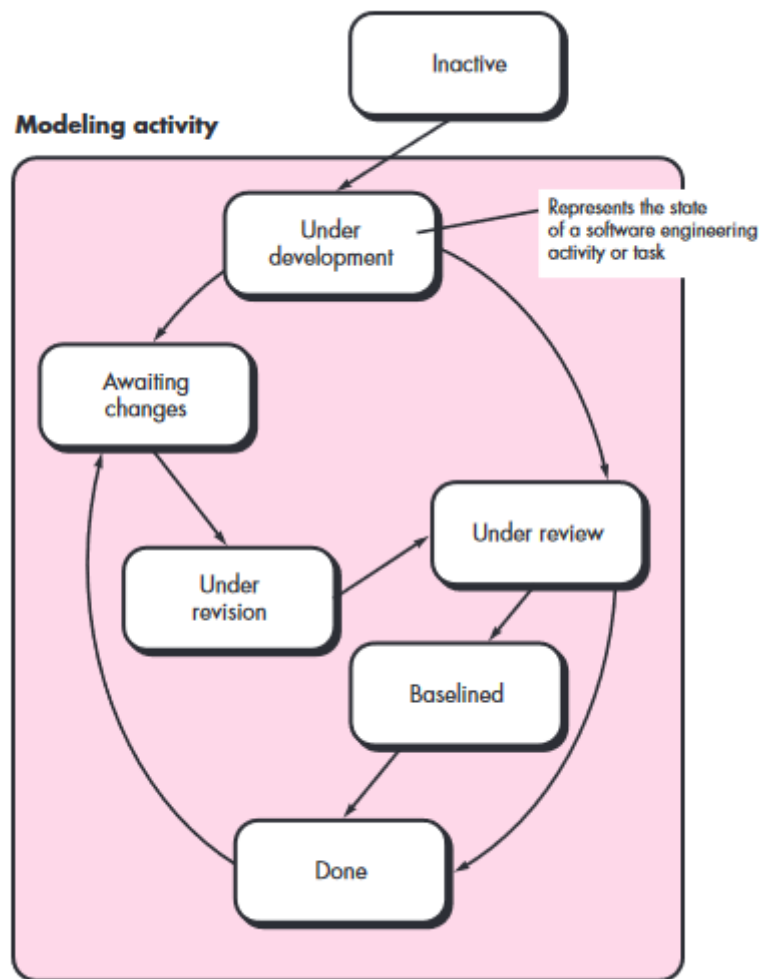


Figure: The concurrent Model

- ✓ The concurrent development model, sometimes called concurrent engineering, allows a software team to represent iterative and concurrent elements of any of the process models.
- ✓ For example, the modeling activity defined for the spiral model is accomplished by invoking one or more of the following software engineering actions: prototyping, analysis, and design.

- ✓ The activity—modeling—may be in any one of the states noted at any given time. Similarly, other activities, actions, or tasks (e.g., communication or construction) can be represented analogously.
- ✓ All software engineering activities exist concurrently but reside in different states.
- ✓ For example, early in a project the communication activity has completed its first iteration and exists in the awaiting changes state. The modeling activity (which existed in the inactive state while initial communication was completed, now makes a transition into the under-development state.
- ✓ If, however, the customer indicates that changes in requirements must be made, the modeling activity moves from the under-development state into the awaiting changes state.
- ✓ Concurrent modeling defines a series of events that will trigger transitions from state to state for each of the software engineering activities, actions, or tasks.
- ✓ For example, during the early stages of design (a major software engineering action that occurs during the modeling activity), an inconsistency in the requirements model is uncovered.
- ✓ This generates the event analysis model correction, which will trigger the requirements analysis action from the done state into the awaiting changes state.
- ✓ Concurrent modeling applies to all types of software development and provides an accurate picture of the current state of a project. Rather than confining software engineering activities, actions, and tasks to a sequence of events, it defines a process network.
- ✓ Each activity, action, or task on the network exists simultaneously with other activities, actions, or tasks. Events generated at one point in the process network trigger transitions among the states.